1.1 NEURAL NETWORKS AND DEEP LEARNING

Deep learning means training neural networks.
Let's say:
Input- size of houses
Output- price of houses corresponding to those prices
I now need a function that predicts the Price of house as a f(size of houses)??
The neuron does this, each neuron receives inputs, performs the computations and then produces an output.

ReLU means rectified Linear unit is responsible for taking a max of 0

Large neural networks are built by stacking several such individual neurons together.

STRUCTURE OF NEURAL NETWORK

1. INPUT
   Eg area, no of bedrooms
2. HIDDEN LAYERS
   Applies activation fn and assigns importance using weights basically does the computation part so they basically learn the patterns that turn the input into features that further help in making output
   Eg hidden layers will learn the relationships like larger area affects the prices, good location means more price etc
3. OUTPUT
   Eg house price
   So basically, neural network is a fn whose internal parameters/ weight are automatically learned from data. We don't tell the computer how to calculate or solve but only feed in egs and let it discover mapping from input to output by adjusting the weights during training.

1.2 SUPERVISED LEARNING WITH NN

Supervised learning is an ML approach where the model learns from labelled data or using examples that already have correct answers so we're repeatedly comparing the model's predictions with the true labels, calculating the error and adjusting the weights.
Types of SL- Regression(predicting continuous values) and classification(predicting discrete categories)

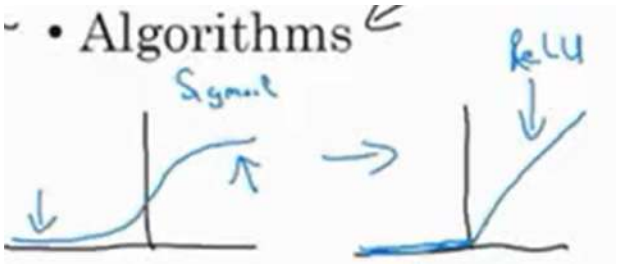
Ex of NN:-
Standard NN
Convolutional NN - used for imaged data
Recurrent NN

Structured Data- each of the features of my database has well defined meaning or a pre-defined format- rows or columns like tables stored in excel or database.

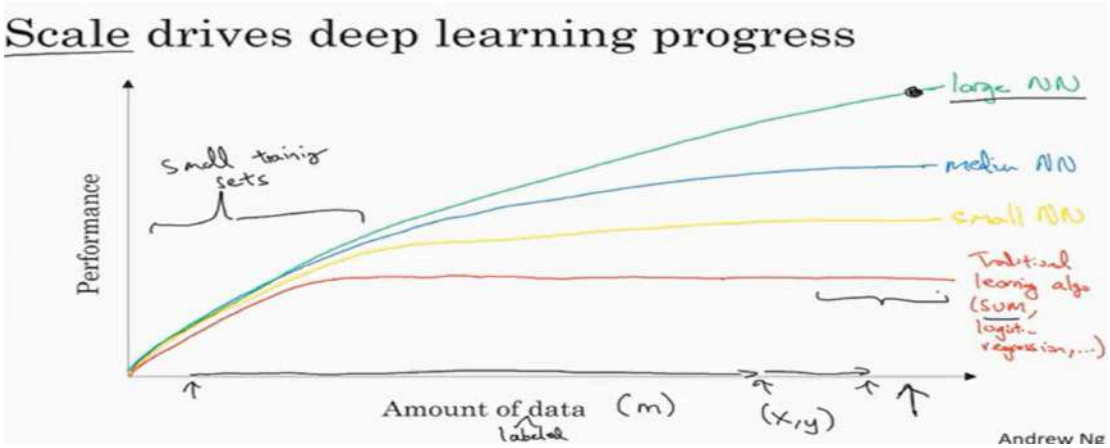
Unstructured Data- no fixed format, eg raw audios or images that have features in the form of pixel values that can't be understood very easily by computers.

1.3 WHY UPSURGE IN DEEP LEARNING?

Unlike the traditional ML algo, deep learning trains better as the training data increases as it learns patterns from examples. Also, now we have GPUs that train faster than CPUs and better algos. For eg switching from sigmoid fn to ReLu fn has made algos like gradient descent work much faster.



From fig, gradient in ReLu fn is =1 while in sigmoid, it approaches 0 which makes the learning rate slower.

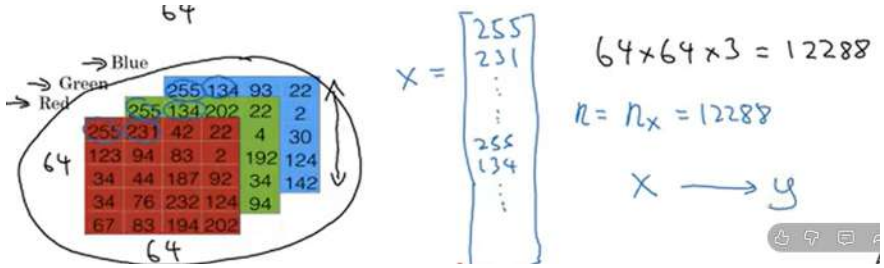


Traditional Machine Learning Performance improves initially but eventually reaches a plateau. Adding more data after that provides little improvement.

Deep Learning Performance continues improving as more data becomes available. DL has ability to scale with data

1.4 Logistics Regression

It is an algo of binary classification. In binary classification, our goal is to learn a classifier that inputs an img represented by a feature vector and predicts whether the corresponding label Y is 0 or 1



12288 here represents the dimension of my feature vector.

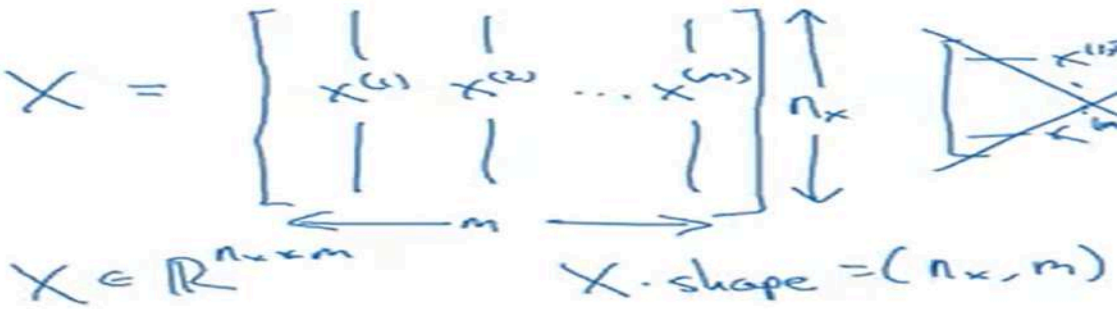
A simple training example denoted by: (x,y),

Where x is an nx dimensional future vector and y is either label 0 or 1
let's say the training set comprises of: m training egs and

training set denoted as: (x1, y1)
Where x1 y1 are the input and output of the 1st training eg

Entire Training set: (x1, y1).....(xm, ym)

To put all the training egs into a compact notation,



$$Y = [\, y^{(1)} \quad y^{(2)} \quad \cdots \quad , y^{(m)} \,]$$

$$Y \in \mathbb{R}^{1 \times m}$$

$$Y.\text{shape} = (1, m)$$

X.shape and Y.shape is the python command for finding the shape of a matrix notation that we use

1.5 (1.4)contd.
yhat is the probability of the chance that y=1 given the input feature x.

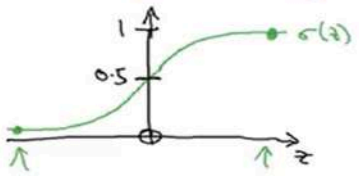
Logistic Regression

Given x, want $\hat{y} = P(y=1|x)$, $0 \le \hat{y} \le 1$

$x \in \mathbb{R}^{n_x}$

Parameters: $w \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$.

Output $\hat{y} = \sigma(w^T x + b)$, where $z = w^T x + b$



$$\sigma(z) = \frac{1}{1+e^{-z}}$$

If z large $\sigma(z) \approx \frac{1}{1+0} = 1$

If z large negative number

$$\sigma(z) = \frac{1}{1+e^{-z}} \approx$$

We thus learn parameters of W and b s.t y hat becomes a good estimate of the chance of Y being=1, to train and learn the parameters w and b, we need a cost function.

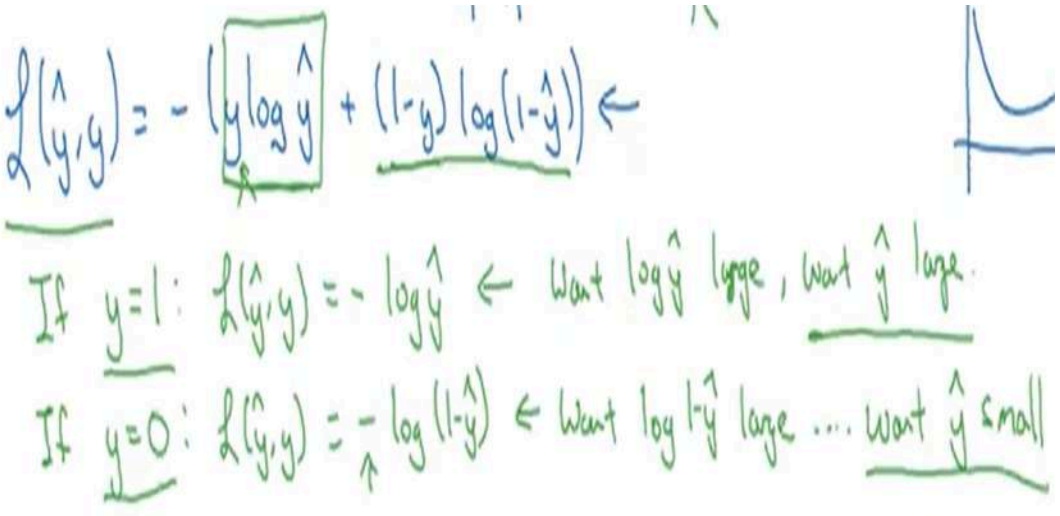
1.6 Logistic Regression Cost function

$$\hat{y}^{(i)} = \sigma(w^T x^{(i)} + b), \text{ where } \sigma(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}} \qquad z^{(i)} = w^T x^{(i)} + b$$

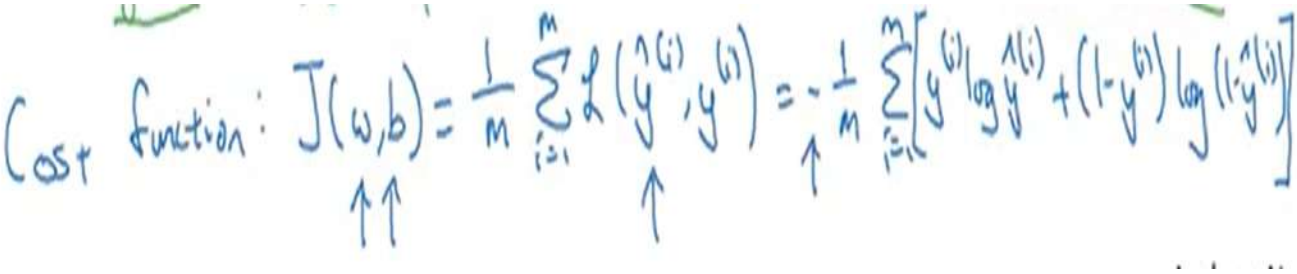
Given $\{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\}$, want $\hat{y}^{(i)} \approx y^{(i)}$.

$x^{(i)}$ $y^{(i)}$ $z^{(i)}$ i-th example.

Loss/error fn to see how well our algo is doing or to compare how good the Algorithm outputs that while the true labels is y



The loss fn tells how good we're doing on 1 training eg for how well we're doing on the **entire training** set we use cost function,



so we have to find w and b to minimise the overall cost function J

1.6 Gradient Descent

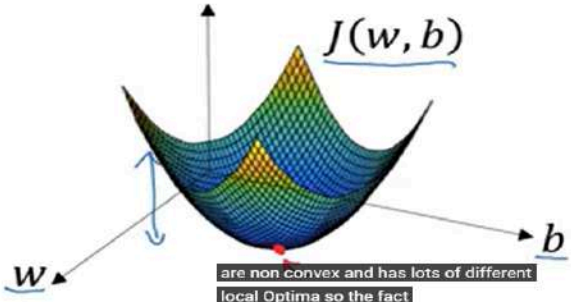
Loss/cost fn told us how well w and b parameters are doing on training set
Gradient descent trains or learns w and b on the training set

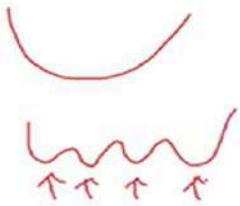
Gradient Descent

Recap: $\hat{y} = \sigma(w^T x + b)$, $\sigma(z) = \frac{1}{1+e^{-z}}$ ←

$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$

Want to find w, b that minimize $J(w,b)$

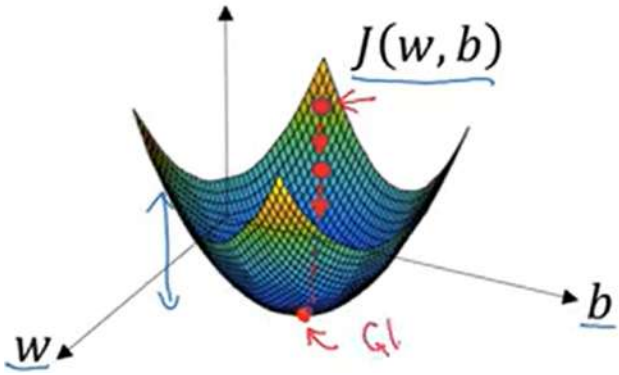




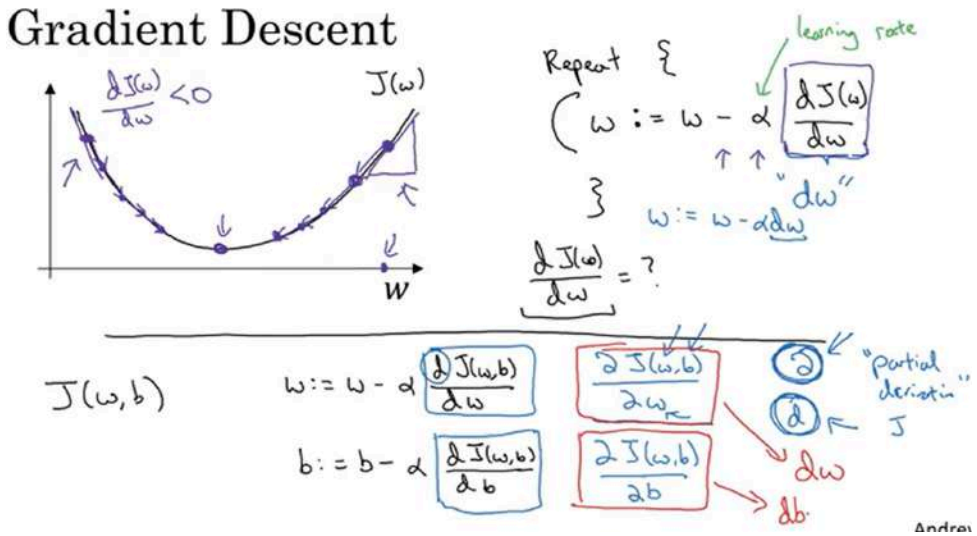
Andrew Ng

The continuous bowl shaped fn is of J(w,b) is convex that has just 1 minima (unlike the other red graph which is non-convex and has several diff local optima).
Thus we first initialise a random value of w and b

After 1 step of gradient descent, the point tries to go in the steepest direction of gradient descent and then after few steps we eventually converge to the global optimum

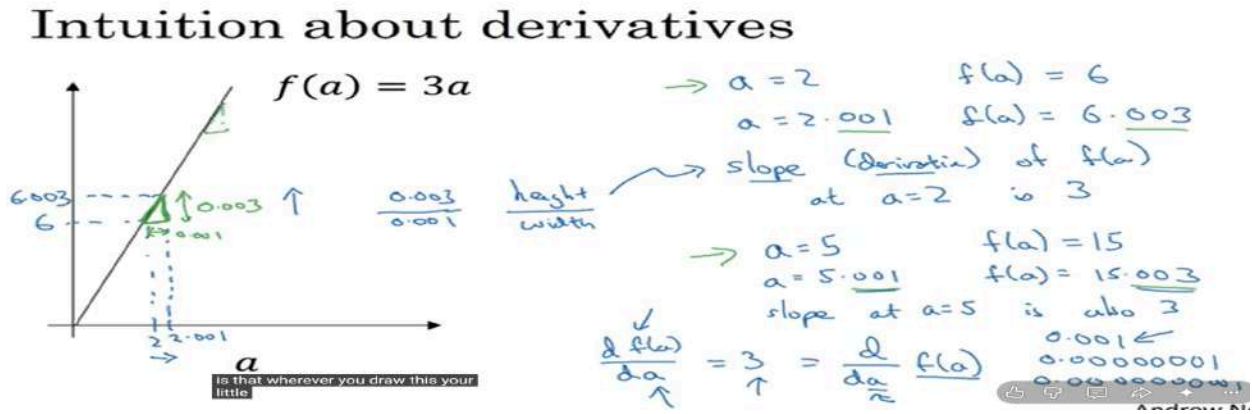


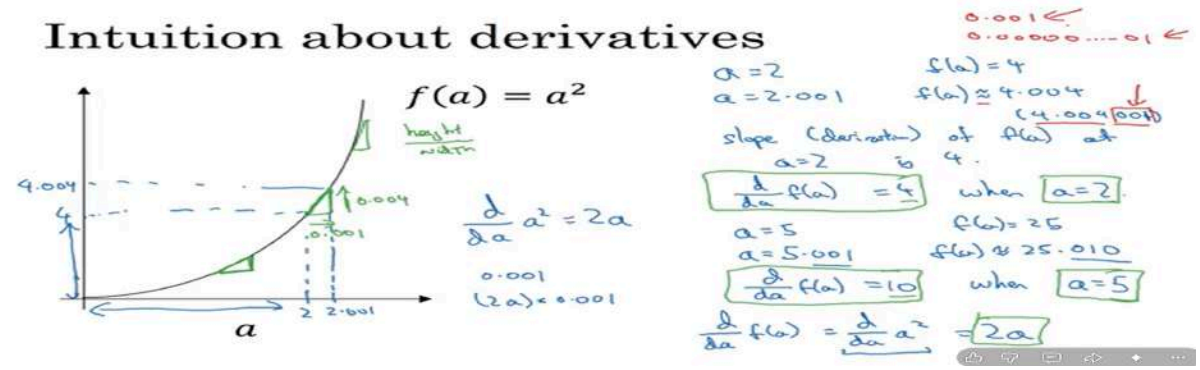
On 2D graph, it is seen as
Alpha is the learning rate that controls how big of a step we take in every iteration

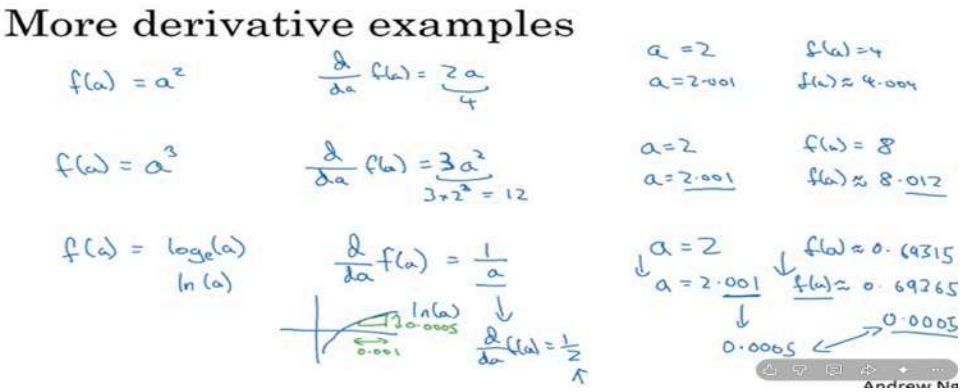


Andrew note that the slope is <0 on the left so w increases and the opposite on the right so w decreases . goal is to go as low as possible on cost fn J and eventually converge at global minima so as to HAVE THE LEAST POSSIBLE COST.

1.6 DERIVATIVES AND EXAMPLES(skip this)

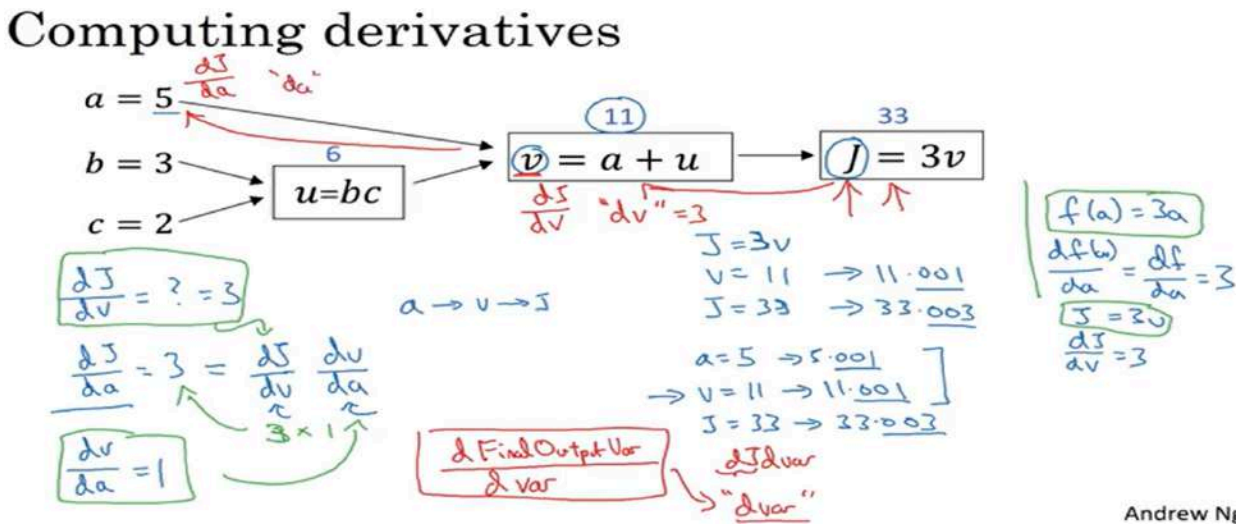






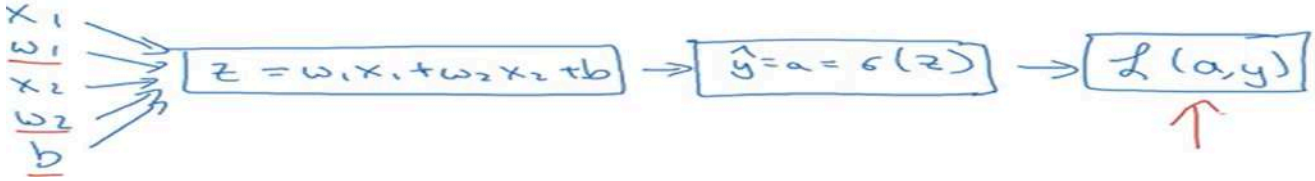
1.7 COMPUTATION GRAPH AND DERIVATIVES

Computation in NN consists of:-
Forward propagation- to compute outputs
Backward - to compute gradients/derivatives
Computation graph explains why it is organised this way
In order to find output, we go from left to right pass
In order to optimise the final output of J, and to find derivatives right to left pass
IN python, we write it as “dvar” means that the derivative of the final output variable J here wrt to the intermediate variables like a, b, c making a chain rule of differentiation as we back propagate

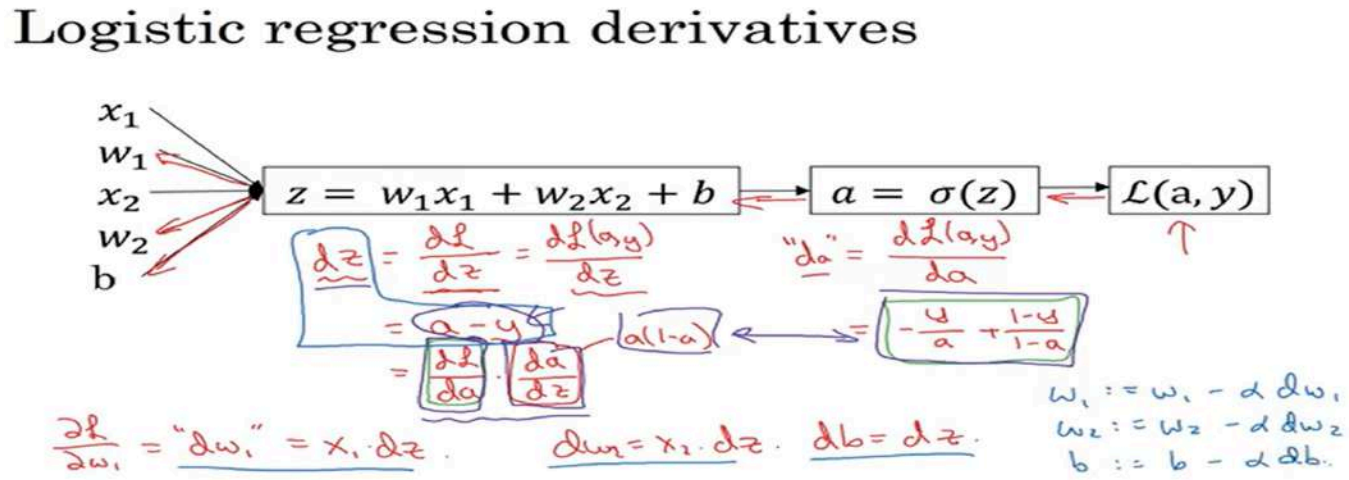


1.7 LOGISTIC REGRESSION GRADIENT DESCENT

$$z = w^T x + b$$
$$\hat{y} = a = \sigma(z)$$
$$\mathcal{L}(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$$

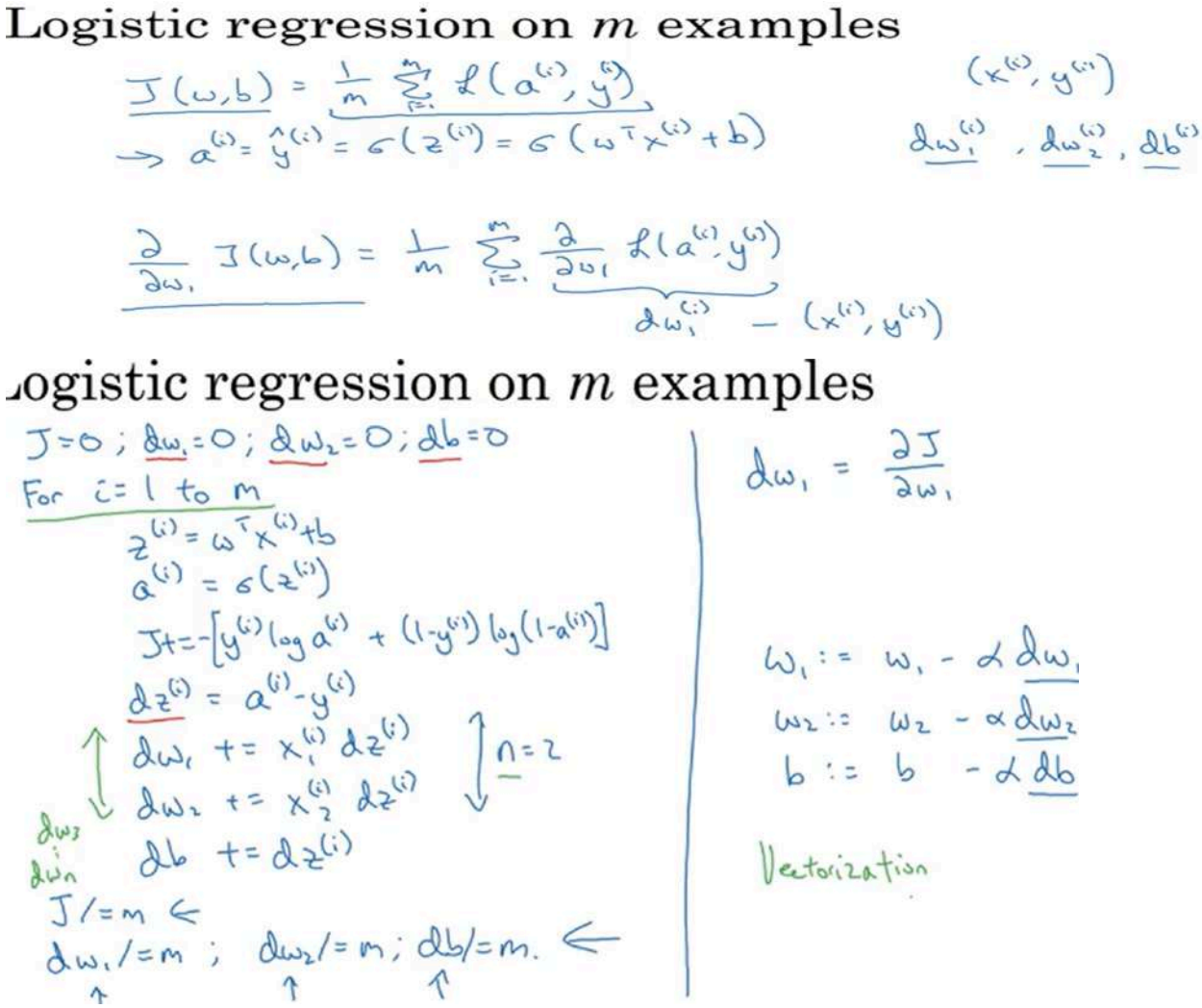


Last step in back propagation is to go back to compute how much to change in w and b and we obtain the updated w and b



FOR M SUCH TRAINING EXAMPLES:-

The ex also assumes that we have just 2 features which is why w1 and w2 only.



PROBLEM- 2 nested for loops
1 for loop for i =1 to m for all m training egs. Inner for loop for all the n features
Vectorization technique resolves this by using numpy or some other library replacing explicit loops with matrix and vector operations.

Vectorization means performing operations on **entire vectors or matrices at once**, instead of processing one element at a time using loops.

Instead of writing:

```
for i in range(n):
    c[i] = a[i] + b[i]
```

we write:

```
c = a + b
```

1. Dot Product

Instead of writing:

```
c = 0
for i in range(n):
   c += a[i] * b[i]
```

he writes:

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = np.dot(a, b)
```

What happens? a = [1 2 3] b = [4 5 6]

Dot product means 1×4 + 2×5 + 3×6= 32 So c = 32

`np.dot()` is simply doing all those multiplications and additions internally, it enables the python library to take better advantage of parallelism

Vectors and matrix valued functions

Say you need to apply the exponential operation on every element of a matrix/vector.

$$v = \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} \rightarrow u = \begin{bmatrix} e^{v_1} \\ e^{v_2} \\ \vdots \\ e^{v_n} \end{bmatrix}$$

```
u = np.zeros((n,1))
for i in range(n):
    u[i]=math.exp(v[i])
```

```
import numpy as np
u = np.exp(v)
```

```
np.log(v)
np.abs(v)
np.maximum(v,0)
v**2
1/v
```

Logistic regression derivatives

```
J = 0, dw1 = 0, dw2 = 0, db = 0
for i = 1 to m:
    z(i) = w^T x(i) + b
    a(i) = σ(z(i))
    J += -[y(i) log ŷ(i) + (1 − y(i)) log(1 − ŷ(i))]
    dz(i) = a(i)(1 − a(i))
    dw1 += x1(i) dz(i)
    dw2 += x2(i) dz(i)
    db += dz(i)
J = J/m, dw1 = dw1/m, dw2 = dw2/m, db = db/m
```

dw = np.zeros((n_x, 1))

dw += x(i) dz(i)

dw /= m

Vectorizing Logistic Regression

$z^{(1)} = w^T x^{(1)} + b$ $\quad a^{(1)} = \sigma(z^{(1)})$

$z^{(2)} = w^T x^{(2)} + b$ $\quad a^{(2)} = \sigma(z^{(2)})$

$z^{(3)} = w^T x^{(3)} + b$ $\quad a^{(3)} = \sigma(z^{(3)})$

$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix} \quad (n_x, m) \quad \mathbb{R}^{n_x \times m}$$

$$w^T \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

$$Z = \begin{bmatrix} z^{(1)} & z^{(2)} & \cdots & z^{(m)} \end{bmatrix} = w^T X + \underbrace{[b \; b \cdots b]}_{1 \times m} = \begin{bmatrix} w^T x^{(1)} + b & w^T x^{(2)} + b & \cdots & w^T x^{(m)} + b \end{bmatrix}$$

$1 \times m$

```
Z = np.dot(w.T, X) + b
```

(1, 1) ℝ "Broadcasting"

$$A = [a^{(1)} \; a^{(2)} \cdots a^{(m)}] = \sigma(Z)$$

2. Matrix-Vector Multiplication

Eg- A = np.array([[1,2], [3,4]]), x = np.array([5,6])

Andrew write y = np.dot(A, x)

First row: 1×5 + 2×6 = 17 Second row : 3×5 + 4×6 = 39

So y = [17 39] NumPy perform for each row multiply add internally.

3. Element-wise Multiplication

a * b (This is **NOT** the dot product.)

Example a = [1,2,3] and b = [4,5,6]

Then a*b gives [4,10,18] ,No addition happens.

4. Broadcasting

a = np.array([1,2,3])
a + 100

Output: [101 102 103]

But how can we add a number to an array? NumPy automatically treats it like [100 100 100] and performs

[1+100, 2+100, 3+100] This feature is called **broadcasting**.

5. Why is Vectorization Faster?

For 1000000 numbers

Loop approach:

for i in range(1000000): do multiplication

Python has to execute the loop one million times.

Instead, np.dot(a,b)

calls highly optimized C/Fortran code underneath, often using SIMD instructions and multiple CPU cores. So instead of Python to NumPy to CPU being repeated a million times, Python makes **1** call and NumPy handles the heavy work efficiently.

6. The Logistic Regression Example

Suppose we have m = 4 examples, n = 3 features

Instead of storing

x^1

x^2

x^3

x^4

separately, we put them into one matrix.

X =[x^1 x^2 x^3 x^4]

Shape becomes (no. of features, n0. of examples) i.e. (3,4)

Weights w have shape (3,1), Then instead of computing

z^1

z^2

z^3

z^4 one by one, we can:-

Z = np.dot(w.T, X) + b,

Shapes w

↓

(3×1)

Transpose

↓

w.T

↓

(1×3)

Multiply (1×3)× (3×4)

↓

(1×4)

So Z= [z^1 z^2 z^3 z^4] All four values are computed **at once**.

A = sigmoid(Z) means A =[a^1 a^2 a^3 a^4] sigmoid is applied to each element of Z automatically.

Then, dZ = A - Y

If A = [0.8 0.2 0.7 0.1], Y = [1 0 1 0]

Then, dZ = [-0.2 0.2 -0.3 0.1] Again, no loop.

Finally,

dw = (1/m) * np.dot(X, dZ.T) instead of

for every example: compute gradient, add it, divide by m

The matrix multiplication computes the sum of gradients for all examples in one operation.

Thus, instead of:

```
for i in range(m):
    z = np.dot(w, X[:, i]) + b
    a = sigmoid(z)
    dz = a - Y[i]
```

```
dw += X[:, i] * dz
```

We write:

```
Z = np.dot(w.T, X) + b
A = sigmoid(Z)
dZ = A - Y
dw = (1/m) * np.dot(X, dZ.T)
db = (1/m) * np.sum(dZ)
```

1.8 Vectorizing Logistic Regression's Gradient Computation

Earlier, for i in range(m):

```
z = np.dot(w.T, X[:, i]) + b
a = sigmoid(z)
dz = a - Y[i]
dw += X[:, i] * dz
db += dz
```

NOW,Suppose

- Number of features = nx=3n_x = 3nx=3
- Number of training examples = m=4m = 4m=4

Instead of storing

x^1
x^2
x^3
x^4

Separately, Andrew stores everything in one matrix.

	Example1	Example2	Example3	Example4
Feature1	x11	x12	x13	x14
Feature2	x21	x22	x23	x24

Feature3 x31 x32 x33 x34
XXX has shape
(3,4) Meaning (number of features, number of examples)This shape convention is used throughout the course.

Step 2: Weight vector

Weights are stored as

www

w

↓

(3,1) Bias b is just one number.

Step 3: Computing Z

Z = np.dot(w.T, X) + b
w shape
(3,1)

Transpose w.T becomes

(1,3).,
(1,3)
×
(3,4)
↓
(1,4)
SO, Z = [z^1 z^2 z^3 z^4]

Instead of computing

z^1
.
.
z^4

one by one, NumPy computes **all four together**.

What about "+ b"? Suppose b = 2, NumPy automatically performs

$[z^1\ z^2\ z^3\ z^4]$
+
[2 2 2 2] using **broadcasting**.
Step 4: Sigmoid
A = sigmoid(Z) Suppose
Z = [2 -1 0 3]

Output becomes : A = [0.881 0.269 0.500 0.953] Again,no loop.

Step 5: Computing dZ
dZ = A - Y, EG:

A = [0.88 0.27 0.50 0.95] and Y = [1 0 1 1]

Then dZ = [-0.12 0.27 -0.50 -0.05]

Step 6: Computing dw
dw = (1/m) * np.dot(X, dZ.T)

(3,1)Exactly the shape we need for dw

Why does this work? Previously we wrote dw = 0

```
for i in range(m):
  dw += x(i) * dz(i)
dw /= m
```

Matrix multiplication performs this accumulation automatically.

So np.dot(X, dZ.T) is simply a fast way of writing

x^1dz^1
+
x^2dz^2
...
+
x^mdz^m
Step 7: Computing db
db = (1/m) * np.sum(dZ)
Suppose
dZ = [-0.2 0.4 -0.1 0.3]

Then np.sum(dZ) returns 0.4 Divide by m to obtain the average. No loop required. Finally,

w = w - alpha * dw b = b - alpha * db

Exactly the same gradient descent rule you learned earlier.Nothing new mathematically.Only the implementation has changed.

Entire vectorized algorithm

Z = np.dot(w.T, X) + b
A = sigmoid(Z)
dZ = A - Y
dw = (1/m) * np.dot(X, dZ.T)
db = (1/m) * np.sum(dZ)
w = w - alpha * dw
b = b - alpha * db This replaces an entire loop over the training examples.

Why is this so much faster?

Imagine m = 1,000,000 examples Loop version:

Example 1
↓
...
↓
Example 1,000,000

Vectorized version: Give all one million examples to NumPy

↓
NumPy performs optimized matrix operations internally
↓
Receive all outputs together. NumPy uses highly optimized low-level libraries, making it much faster than looping in Python

Broadcasting Rules (Simplified)

NumPy compares dimensions starting from the **last dimension**.Two dimensions are compatible if:

- They are equal.
- One of them is 1.

If these conditions are satisfied, broadcasting can occur.Broadcasting is a feature in NumPy that allows arrays of different shapes to participate in arithmetic operations.Instead of physically copying data, NumPy behaves **as if** the smaller array has been expanded to match the larger one.

f we want to divide every column by its total calories to compute percentages, we can simply write

A / column_sum

instead of looping through every row and column.Broadcasting automatically expands the smaller array to match the matrix dimensions.Broadcasting is **very powerful**, but it can also hide bugs. For example:

A.shape = (3,4)
B.shape = (4,)

NumPy may still broadcast them, but not always in the way you intended so **check the shapes** of arrays frequently.

print(X.shape)
print(w.shape)
print(b)

to make sure every matrix has the expected dimensions. **always write the shape beside every variable**. EG:

X → (n_x, m) w → $(n_x, 1)$ b → scalar Z → (1, m) A → (1, m) Y → (1, m) dZ → (1, m) dw → $(n_x, 1)$ db → scalar

```
In [1]: import numpy as np

        a = np.random.randn(5)

In [2]: print(a)

[ 0.50290632 -0.29691149  0.95429604 -0.82126861 -1.46269164]

In [3]: print(a.shape)

(5,)

In [4]: print(a.T)

[ 0.50290632 -0.29691149  0.95429604 -0.82126861 -1.46269164]

In [5]: print(np.dot(a,a.T))

4.06570109321
```

This is called a Rank 1 array i.e. neither a column nor row vector, so don't use such data structures, specify it properly as in next example with a comma.

```
a = np.random.randn(5,1)
print(a)

[[-0.0967311 ]
 [-2.38617377]
 [-0.3243588 ]
 [-0.96216349]
 [ 0.54410384]]

print(a.T)

[[-0.0967311  -2.38617377 -0.3243588  -0.96216349  0.54410384]]

print(np.dot(a,a.T))

[[ 0.00935691  0.23081721  0.03137558  0.09307113 -0.05263176]
 [ 0.23081721  5.69382526  0.77397645  2.29588928 -1.2983263 ]
 [ 0.03137558  0.77397645  0.10520863  0.31208619 -0.17648487]
 [ 0.09307113  2.29588928  0.31208619  0.92575858 -0.52351684]
 [-0.05263176 -1.2983263  -0.17648487 -0.52351684  0.29604898]]
```

```
a = np.random.randn(5)
    a.shape = (5,)
    "rank 1 array"
```
Don't use

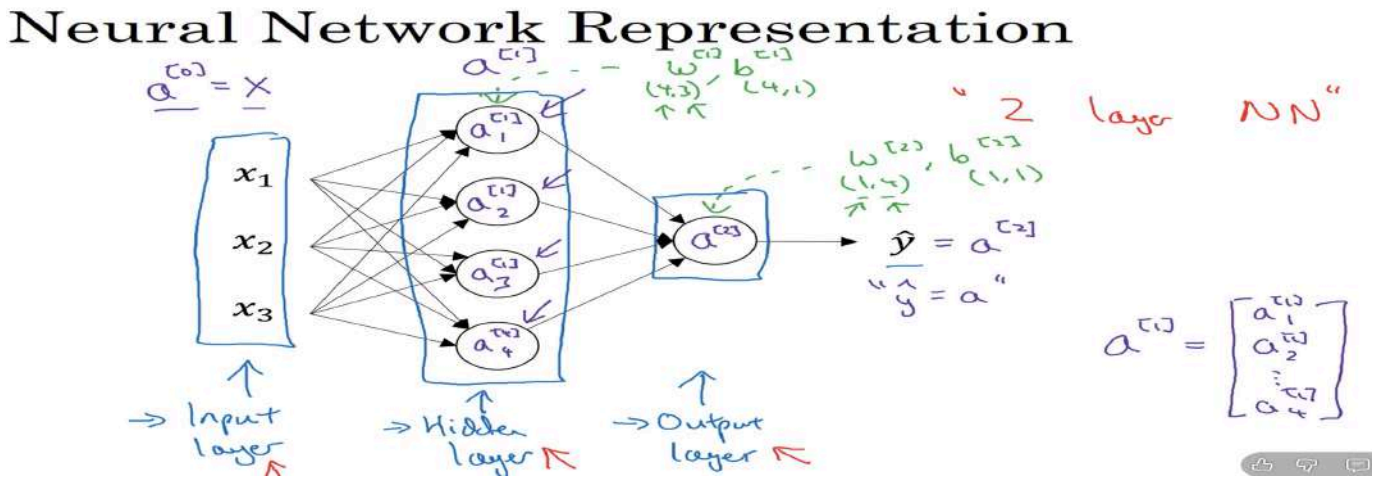
```
a = np.random.randn(5,1) → a.shape = (5,1)   column vector ✓
a = np.random.randn(1,5) → a.shape = (1,5)   row vector ✓
assert(a.shape == (5,1)) ←
    a = a.reshape((5,1))
```

Using Assertions checking shapes explicitly. Example:
assert(w.shape == (3,1))

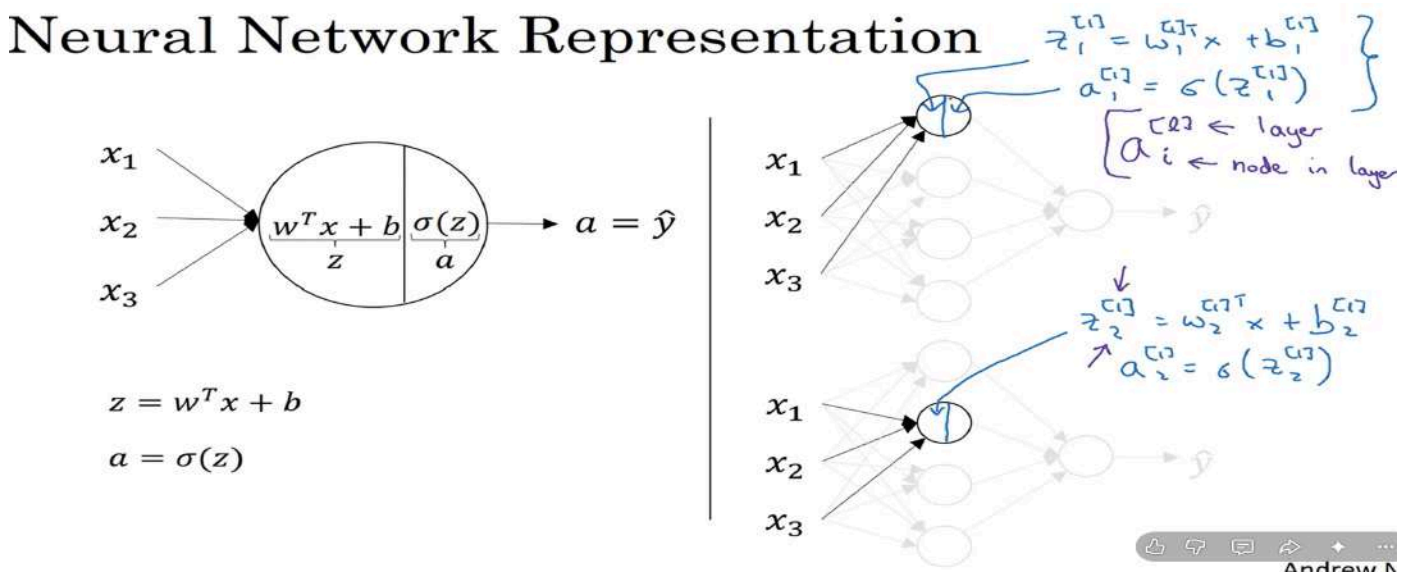
If the shape is incorrect, Python raises an error immediately.This helps detect bugs early.

Reshaping Arrays If you accidentally create a rank-1 array,
a = np.array([1,2,3]) you can convert it into a column vector using
a = a.reshape((3,1)) So, a.shape returns (3,1)

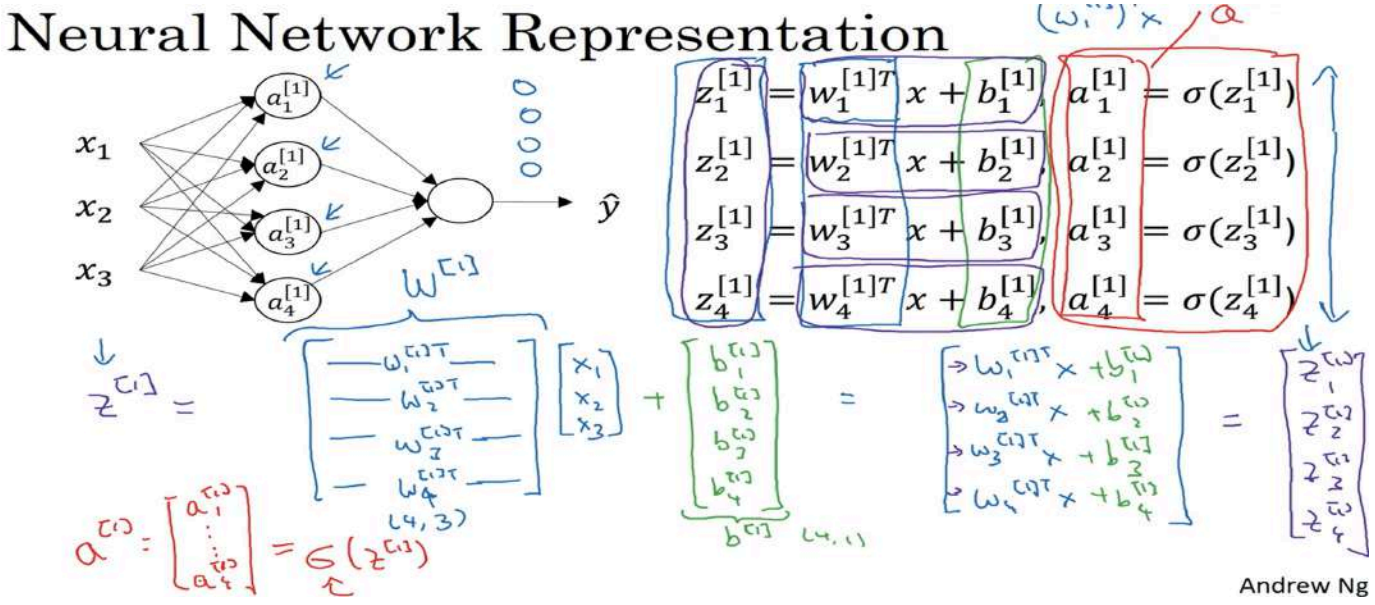
2.1 Neural Network Activation refers to the values that the different layers in the neural network are passing to the subsequent layers

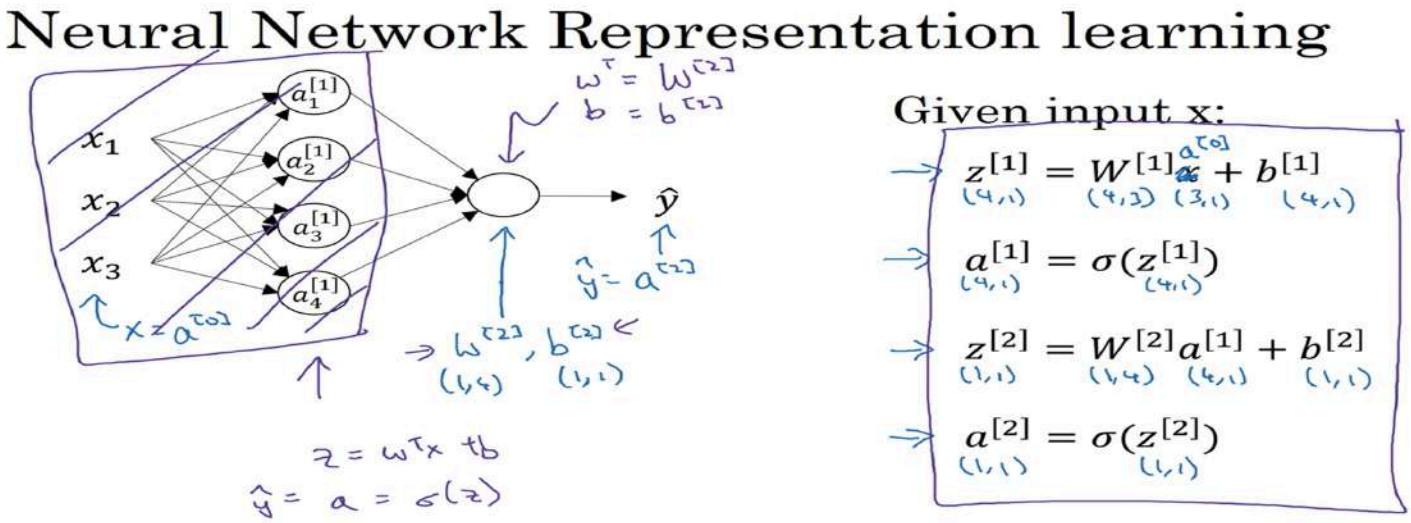


2 layer NN because input layers are not officially counted or taken as 0 layer. hidden layer has its own parameters w and b, dim 4,4 since 3 input features n 4 nodes.



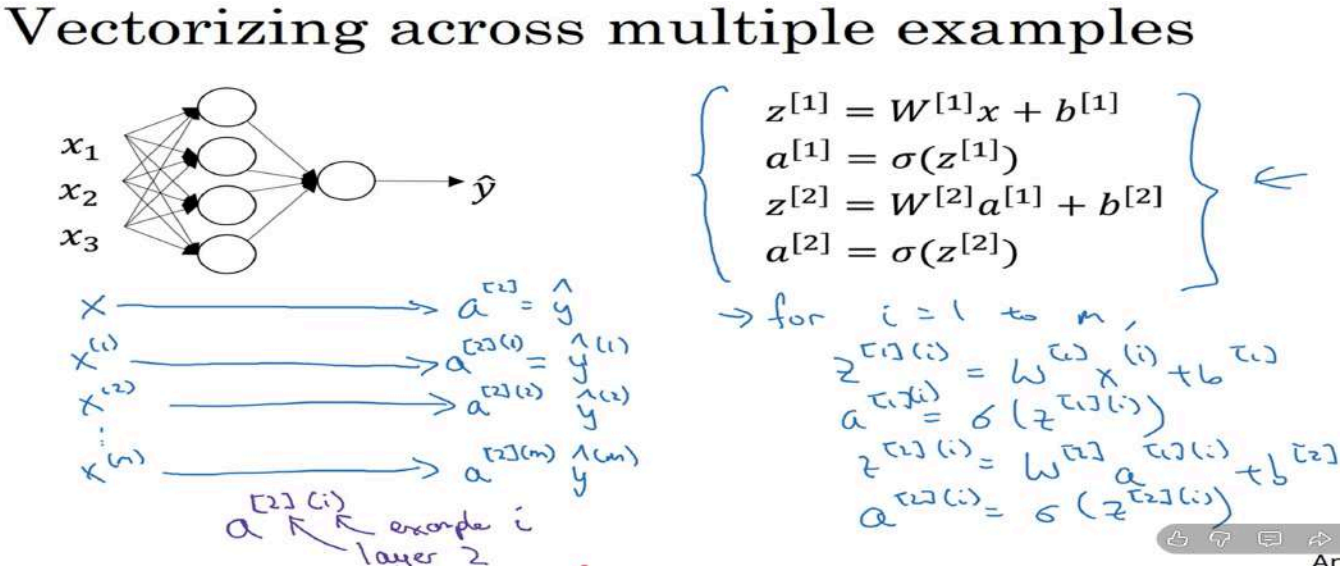
Above fig had logistic regression in the left.





What does (n^{[l]}) represent?The no. of neurons(units) in layer (l).

For m such training exs.

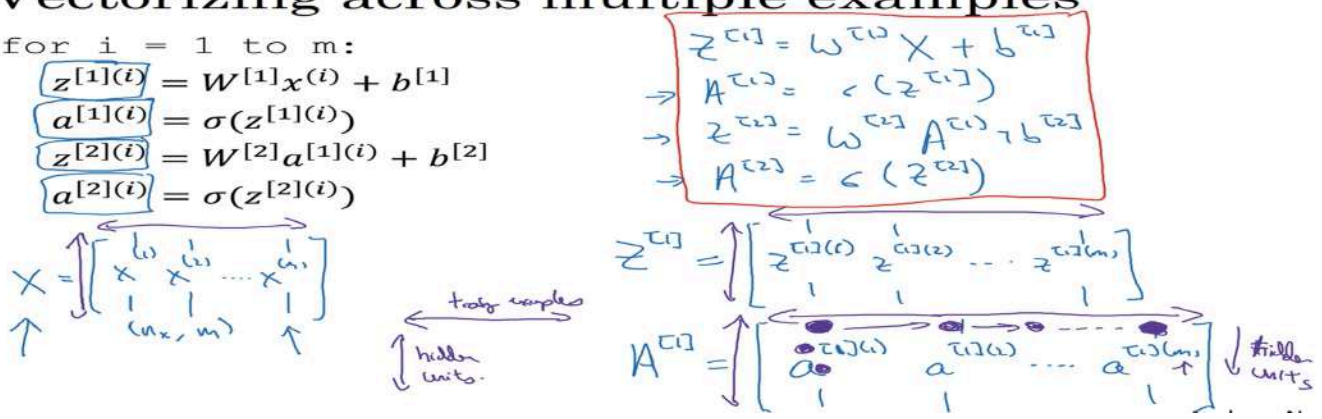


In the below fig, matrices such as z and a have, horizontally we index across the training egs so horizontal index corresponds to the training examples and vertically, vertical index refers to the diff nodes in the NN

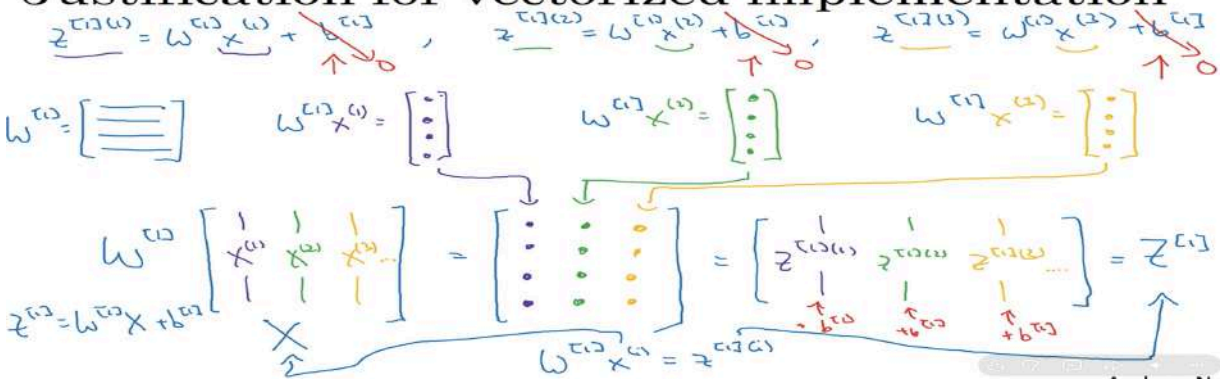
Vectorizing across multiple examples

```
for i = 1 to m:
```

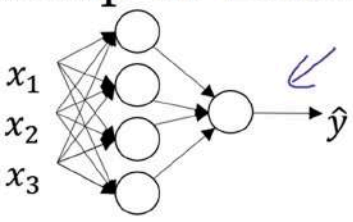
$$z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]}$$
$$a^{[1](i)} = \sigma(z^{[1](i)})$$
$$z^{[2](i)} = W^{[2]}a^{[1](i)} + b^{[2]}$$
$$a^{[2](i)} = \sigma(z^{[2](i)})$$



Justification for vectorized implementation



Recap of vectorizing across multiple examples



$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

$$A^{[1]} = \begin{bmatrix} | & | & & | \\ a^{[1](1)} & a^{[1](2)} & \cdots & a^{[1](m)} \\ | & | & & | \end{bmatrix}$$

```
for i = 1 to m
```

$$z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]}$$
$$a^{[1](i)} = \sigma(z^{[1](i)})$$
$$z^{[2](i)} = W^{[2]}a^{[1](i)} + b^{[2]}$$
$$a^{[2](i)} = \sigma(z^{[2](i)})$$

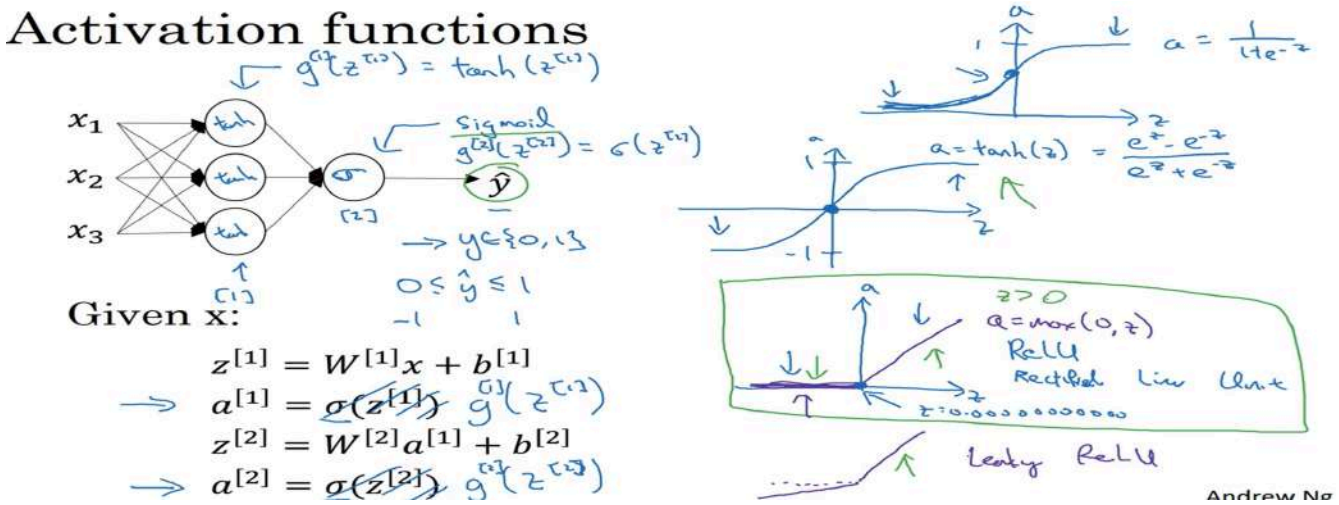
$A^{[0]} = X = a^{[0]}$ $\quad x^{(i)} = a^{[0](i)}$

$$Z^{[1]} = W^{[1]}X + b^{[1]} \leftarrow W^{[1]}A^{[0]} + b^{[1]}$$
$$A^{[1]} = \sigma(Z^{[1]})$$
$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$
$$A^{[2]} = \sigma(Z^{[2]})$$

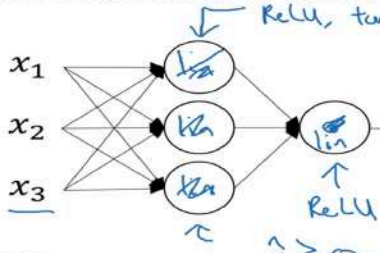
Andrew N

2.2 ACTIVATION FUNCTION

Disadvantage of Tanh and Sigmoid fn:- If Z is very large or small, the slope of the fn becomes very small, as in graph, the line is no longer curve but mostly a straight line, this is vanishing gradient descent.



Activation function



$y \in \mathbb{R}$

\$0 ..... \$1,000,000s

$\hat{y} \geq 0$

$a^{[1]} = z^{[1]} = w^{[1]}x + b^{[1]}$

$a^{[2]} = z^{[2]} = w^{[2]}a^{[1]} + b^{[2]}$

$a^{[2]} = w^{[2]}(w^{[1]}x + b^{[1]}) + b^{[2]}$

$= (w^{[2]}w^{[1]})x + (w^{[2]}b^{[1]} + b^{[2]})$

$= w'x + b'$

Given x:

$z^{[1]} = W^{[1]}x + b^{[1]}$

$a^{[1]} = g^{[1]}(z^{[1]}) = z^{[1]}$

$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$

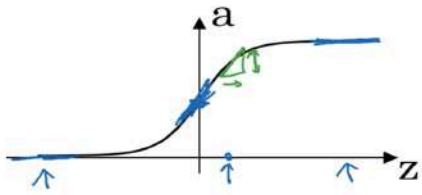
$a^{[2]} = g^{[2]}(z^{[2]}) = z^{[2]}$

$g(z) = z$

"linear activation function"

$g(z) = z$

Sigmoid activation function



$$g(z) = \frac{1}{1+e^{-z}}$$

$a = g(z) = \frac{1}{1+e^{-z}}$

$g'(z) = \frac{d}{dz}g(z)$ = slope of $g(x)$ at z

$= \frac{1}{1+e^{-z}}\left(1 - \frac{1}{1+e^{-z}}\right)$

$= g(z)\left(1 - g(z)\right)$ ←

$= a(1-a)$

$g'(z) = a(1-a)$

$z = 10$. $g(z) \approx 1$

$\frac{d}{dz}g(z) \approx 1(1-1) \approx 0$

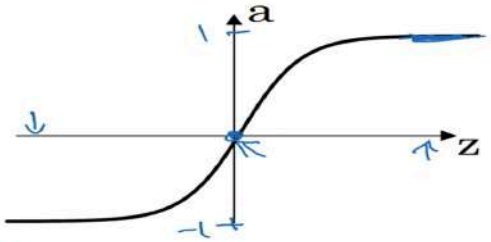
$z = -10$ $g(z) \approx 0$

$\frac{d}{dz}g(z) \approx 0 \cdot (1-0) \approx 0$

$z = 0$ $g(z) = \frac{1}{2}$

$\frac{d}{dz}g(z) = \frac{1}{2}\left(1 - \frac{1}{2}\right) = \frac{1}{4}$

Tanh activation function



$$g(z) = \tanh(z)$$

$$= \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

$g'(z) = \frac{d}{dz}g(z)$ = slope of $g(z)$ at z

$= 1 - (\tanh(z))^2$ ←

$a = g(z)$, $g'(z) = 1 - a^2$

$z = 10$ $\tanh(z) \approx 1$

$g'(z) \approx 0$

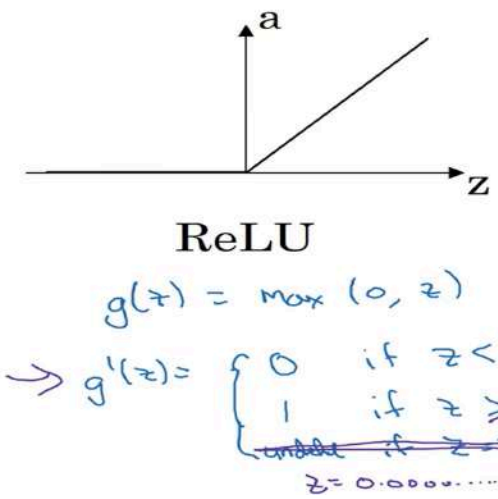
$z = -10$ $\tanh(z) \approx -1$

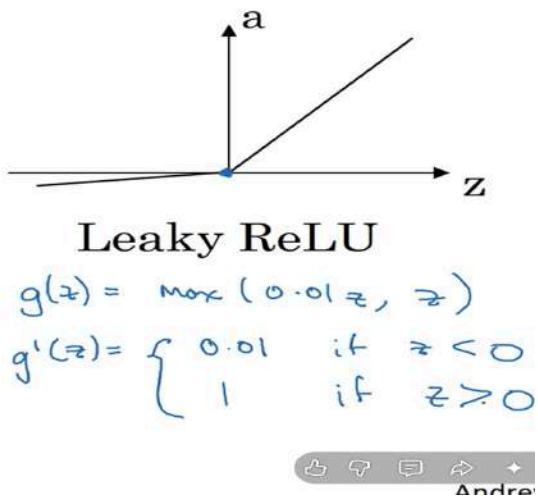
$g'(z) \approx 0$

$z = 0$ $\tanh(z) = 0$

$g'(z) = 1$

ReLU and Leaky ReLU





[z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}]

[a^{[l]} = g(z^{[l]})]

The function [g(z)] is called the **activation function**. Suppose every layer performs only [z = Wx+b]with **no activation function**.Even if we stack many layers,the entire network is still equivalent to one linear transformation.A deep network without activation functions behaves like logistic regression (except possibly the final sigmoid for binary classification).Activation functions allow neural networks to learn **complex nonlinear relationships**.

1. Sigmoid Activation

Outputs probabilities mainly for binary classification output.

BUT Saturates for very large or very small inputs and can cause the **vanishing gradient** problem and not zero-centered.

2. Tanh Activation

Zero-centered and Often trains faster than sigmoid but still suffers from vanishing gradients.

3. ReLU (Rectified Linear Unit)

fast to compute and does not saturate for positive values

Disadvantages Can suffer from the **dying ReLU** problem if neurons always receive negative inputs.

4. Leaky ReLU

Instead of outputting zero for negative inputs, it outputs a small negative value thus Reduces the dying ReLU problem, gradients flow even for negative inputs.

Output Layer (Multi-class Classification)Softmax, which outputs a probability distribution across multiple classes.

Comparison Table

Activation	Output Range	Common Use
Sigmoid	(0,1)	Binary classification output
Tanh	(-1,1)	Hidden layers (older networks)
ReLU	[0,∞)	Most hidden layers
Leaky ReLU	(-∞,∞)	Hidden layers (avoids dying ReLU)

Sigmoid can’t be used in hidden layers Because it can cause vanishing gradients and slows training.

1. Why can't we use only linear activation functions? Because multiple linear layers collapse into a single linear transformation, preventing the network from learning complex nonlinear relationships.Linear Activation

Returns the input unchanged:[g(z)=z]

2. Why are nonlinear activation functions important?They enable neural networks to model complex decision boundaries and hierarchical feature representations.

3. When is a linear activation function appropriate?Typically in the output layer of regression models where the target can take any real value.

Derivatives of Activation Functions

During forward propagation, each neuron computes:

[z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}]

[a^{[l]} = g(z^{[l]})]

Why Are These Derivatives Important?

During backpropagation, we need to know: **How does the activation change when the input (z) changes?** by the **derivative** of the activation function. gradients are computed using the Chain Rule. At each layer, the derivative of the activation function determines how much of the gradient flows backward.Without these derivatives, the network cannot update its weights correctly.

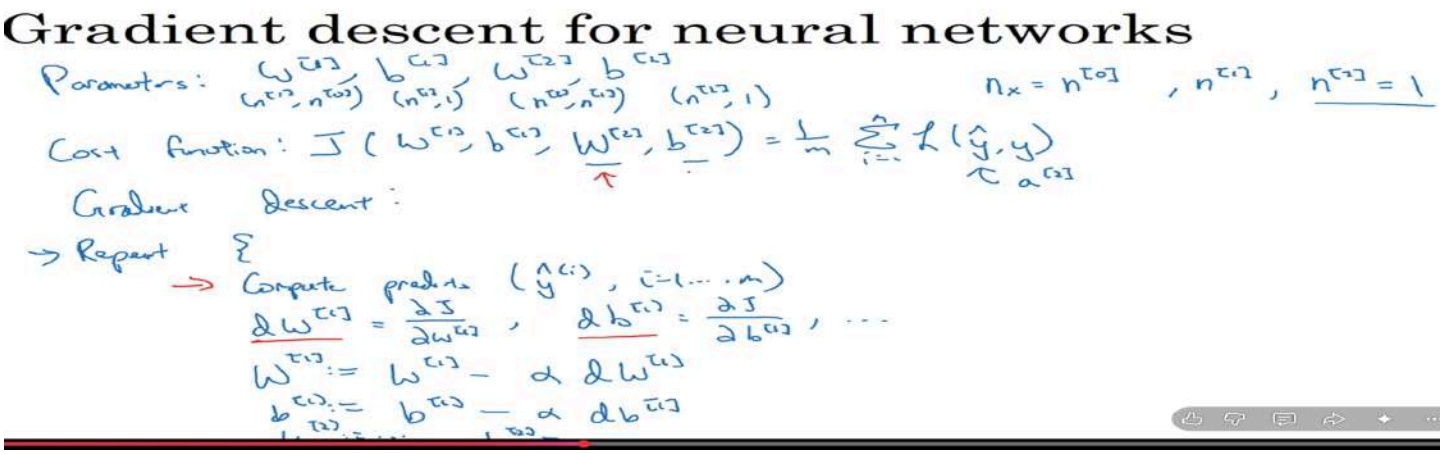
- Sigmoid derivative: (a(1-a)).
- Tanh derivative: (1-a^2).
- ReLU derivative is simple: 0 or 1.
- Leaky ReLU keeps a small gradient for negative inputs.

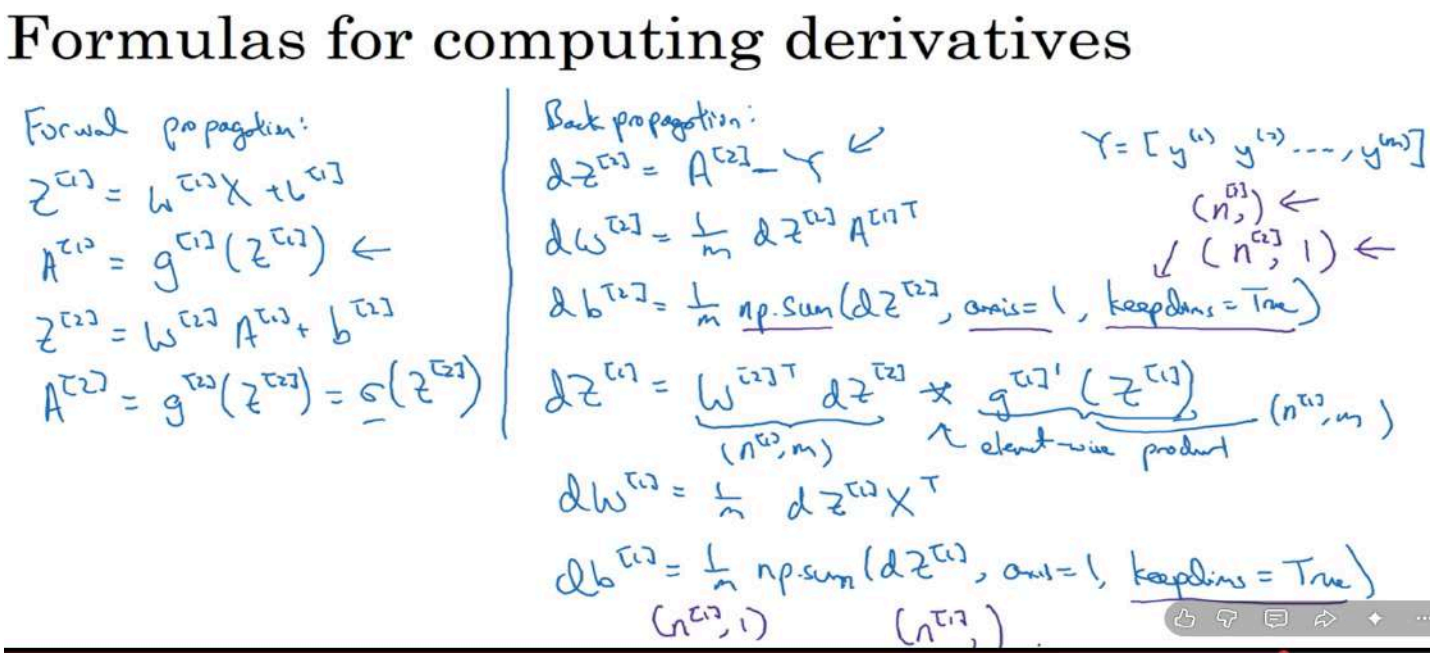
1. Why do we need derivatives of activation functions?Because backpropagation uses them to compute gradients and update the model parameters.

3. Why is ReLU computationally efficient?Its derivative is extremely simple: it is 0 for negative inputs and 1 for positive inputs.

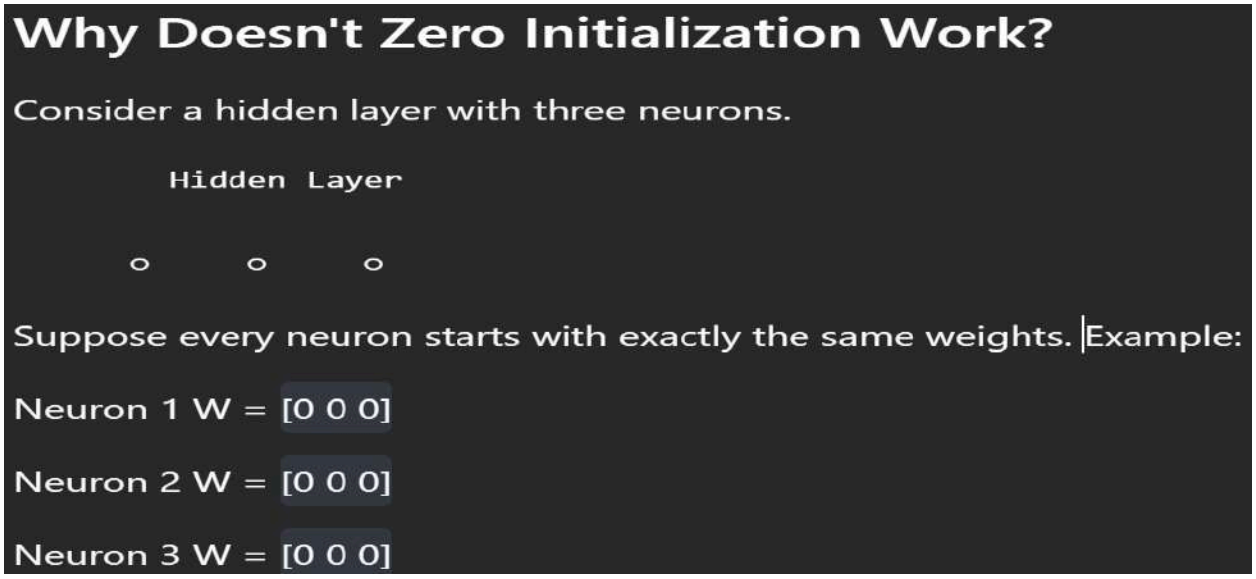
4. Why does Leaky ReLU reduce the dying ReLU problem?Because its derivative for negative inputs is a small positive constant (e.g., 0.01), allowing gradients to continue flowing.

2.2 Gradient Descent of Neural Network





2.2 Random Initialisation



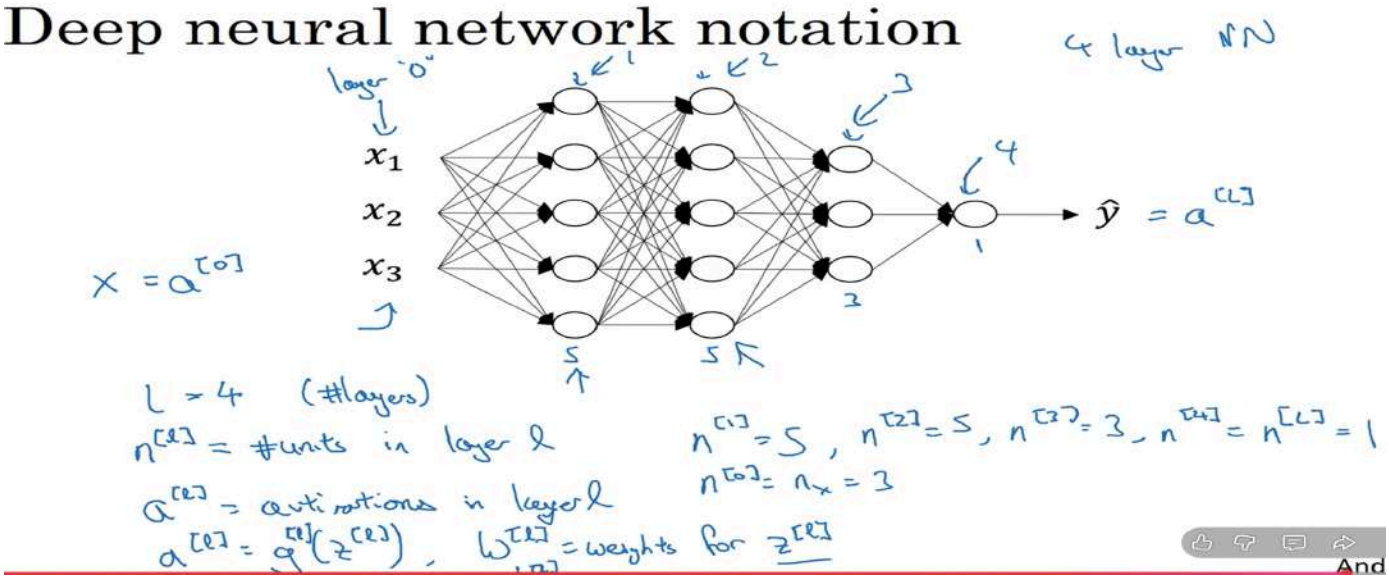
Since the weights are identical, each neuron receives the same input, computes the same output,and receives the same gradient. Therefore, every update is identical.The neurons never become different. they continue learning exactly the same thing.Instead of learning different features,all neurons become copies of one another.This defeats the purpose of having multiple neurons called symmetry problem**.

Each neuron starts different,learns different features,and contributes unique information.

The multiplication by 0.01 keeps initial weights small. Large initial weights can produce very large activations, especially with sigmoid or tanh, causing saturation and slowing learning. (the graph becomes straight for large n

small Z). Unlike weights,biases can safely be initialized to zero if we already started with diff w.

2.3 Deep L- Layer Neural Network



For every layer:

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$$

$$A^{[l]} = g^{[l]}(Z^{[l]})$$

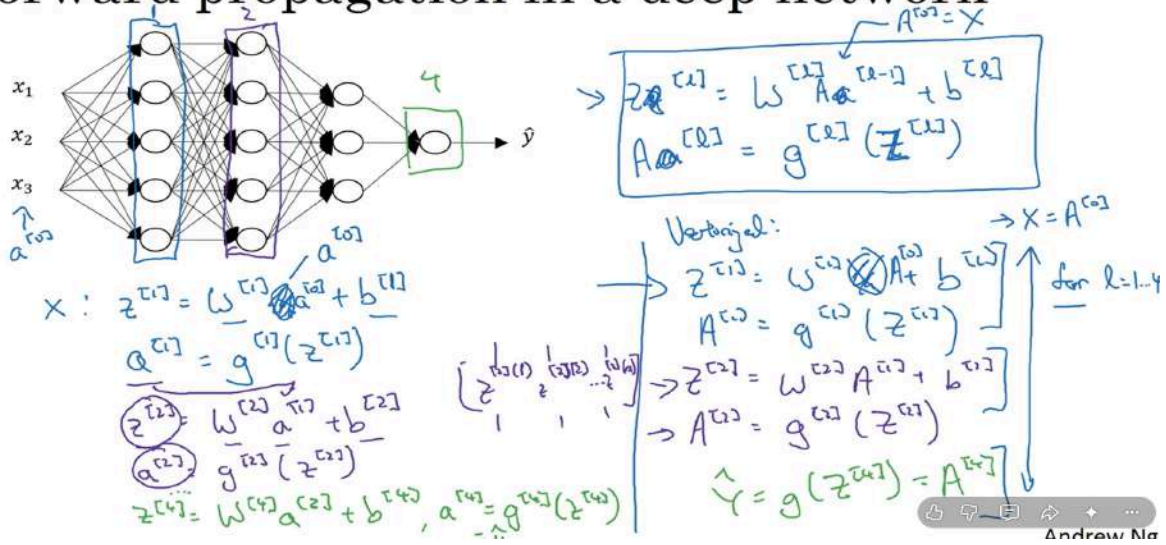
The superscript $[l]$ simply means "this belongs to layer l." For example:

- $W^{[1]}$: weights of the first hidden layer.
- $W^{[2]}$: weights of the second hidden layer.
- $W^{[L]}$: weights of the output layer.

2.4 Forward Propagation in Deep Network

Instead of writing Separate equations for every layer, we use a loop because every layer performs the same computation.

Forward propagation in a deep network



For every layer:

Weight Matrix

(n^{[l]},n^{[l-1]})]

Bias

(n^{[l]},1)]

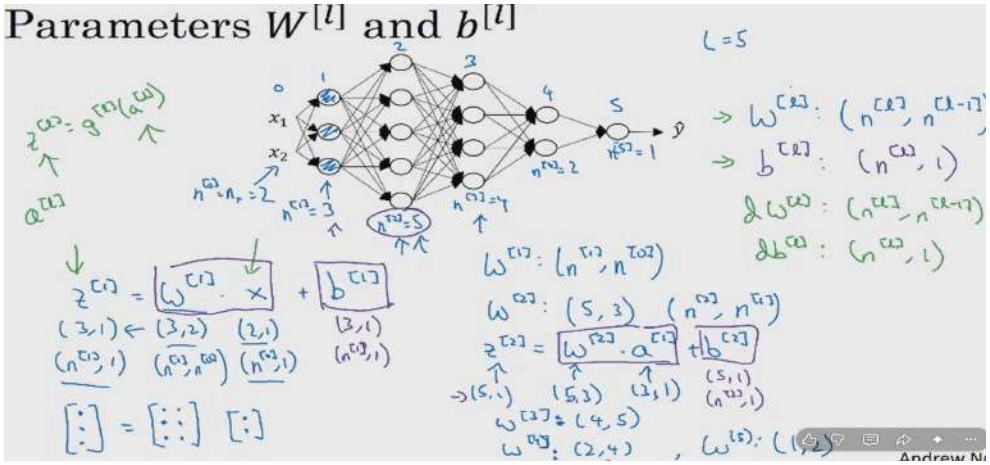
Activations

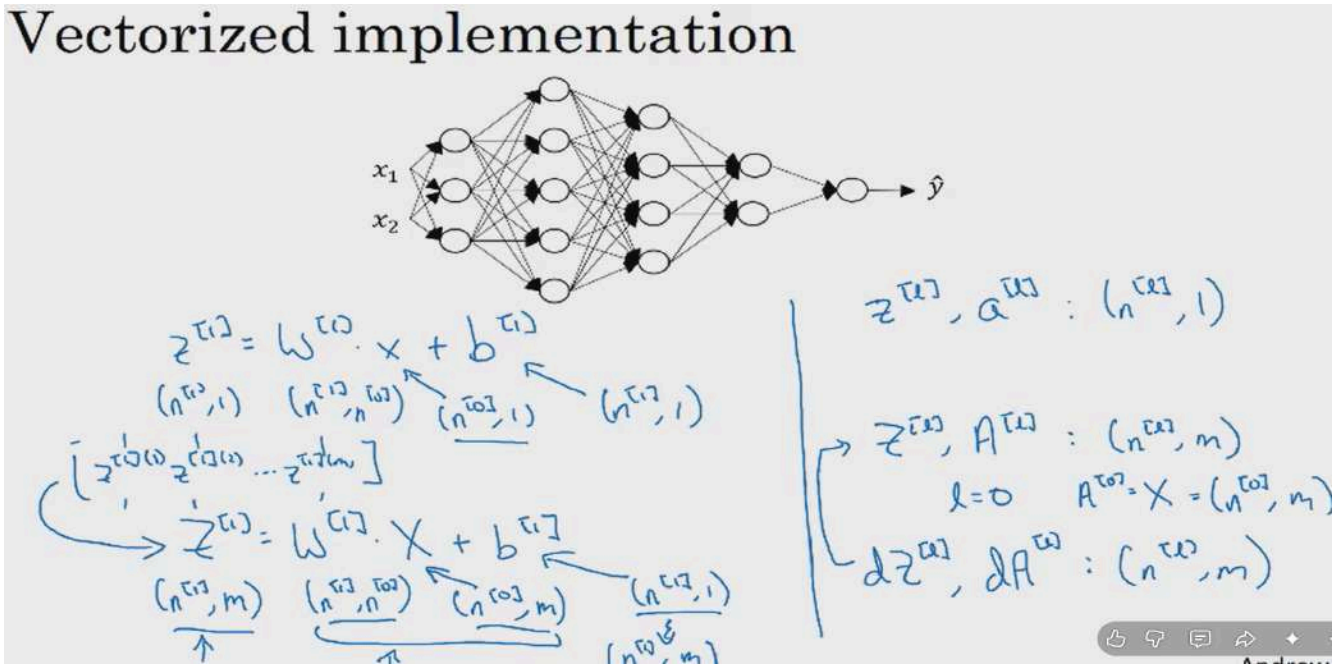
(n^{[l]},m)]

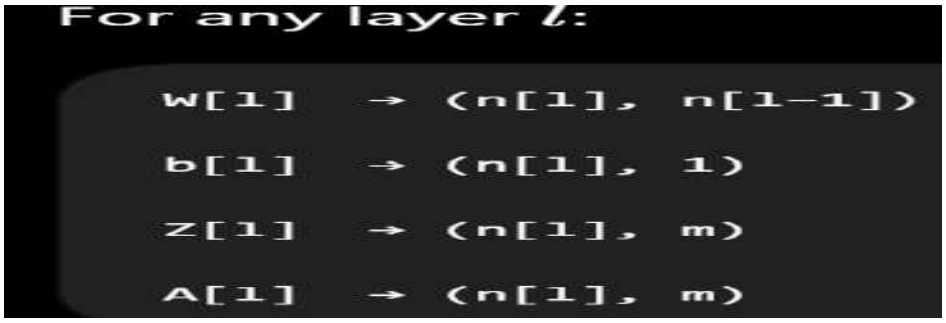
Linear Output

(n^{[l]},m)]

Parameters $W^{[l]}$ and $b^{[l]}$







Variable	Shape
$X = A^0$	(5,100)
W^1	(4,5)
b^1	(4,1)
Z^1	(4,100)
A^1	(4,100)
W^2	(3,4)
b^2	(3,1)
Z^2	(3,100)
A^2	(3,100)
W^3	(1,3)
b^3	(1,1)
Z^3	(1,100)
A^3	(1,100)

for 100 training egs.

Forward propagation for layer l

→ Input $a^{[l-1]}$

→ Output $a^{[l]}$, cache ($z^{[l]}$)

$$z^{[l]} = W^{[l]} \cdot a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g^{[l]}(z^{[l]})$$

Vectorized:

$$Z^{[l]} = W^{[l]} \cdot A^{[l-1]} + b^{[l]}$$

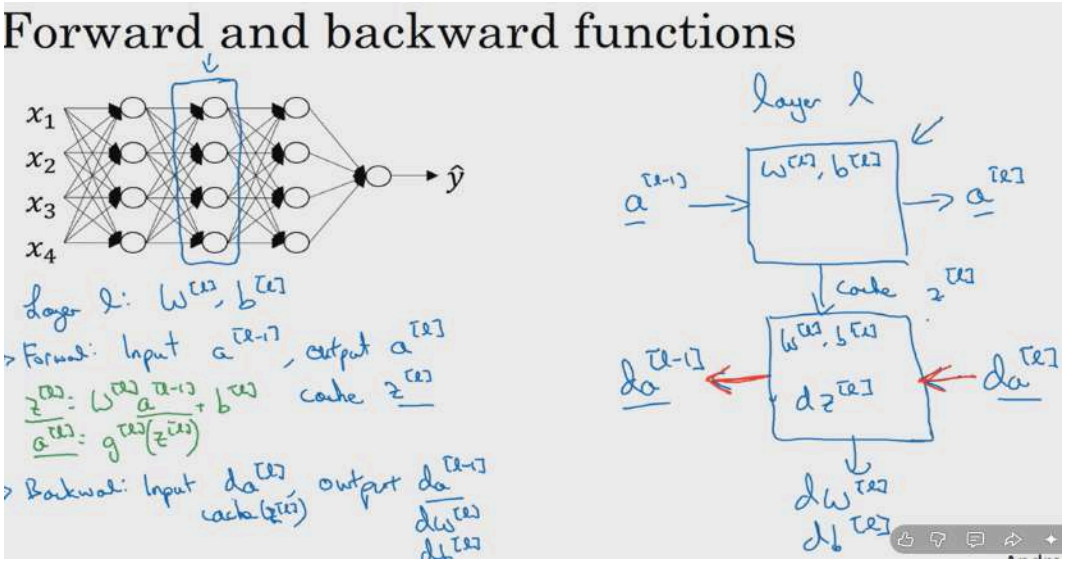
$$A^{[l]} = g^{[l]}(Z^{[l]})$$

$a^{[0]}$ = x

$A^{[0]}$

$X = A^{[0]} \rightarrow [\,] \rightarrow [\,] \rightarrow [\,] \rightarrow$

Forward and backward functions



Layer l: $W^{[l]}, b^{[l]}$

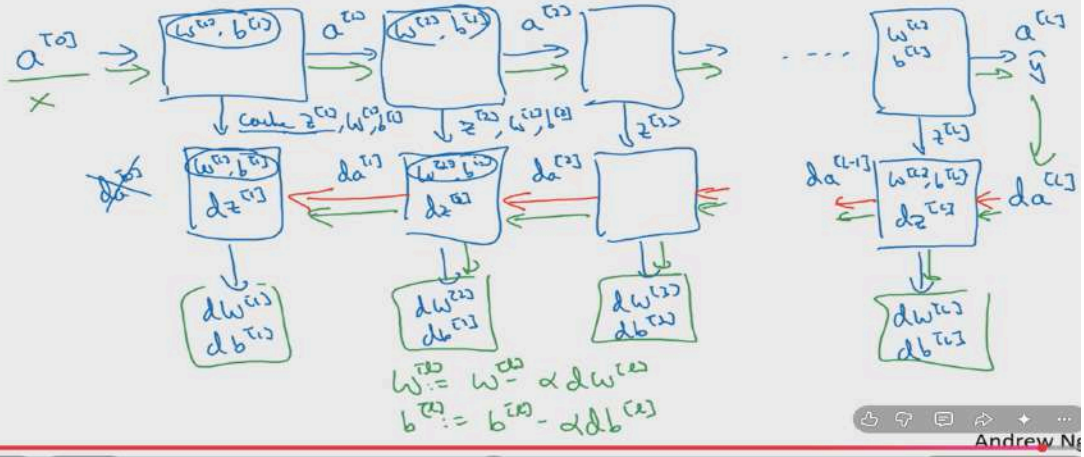
> Forward: Input $a^{[l-1]}$, output $a^{[l]}$

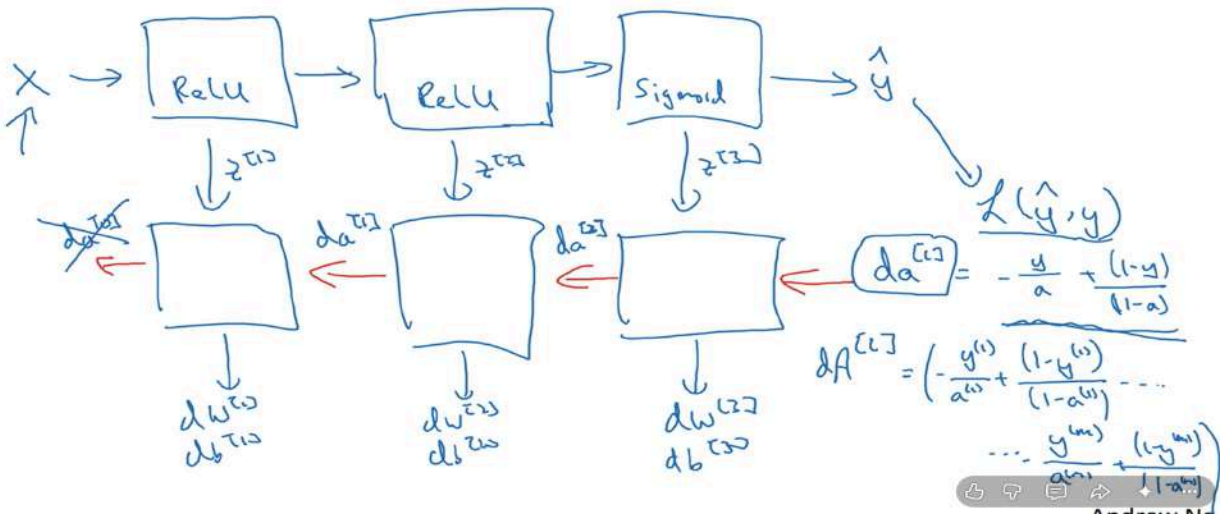
$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$ cache $z^{[l]}$

$a^{[l]} = g^{[l]}(z^{[l]})$

> Backward: Input $da^{[l]}$, cache ($z^{[l]}$) output $da^{[l-1]}$, $dW^{[l]}$, $db^{[l]}$

Forward and backward functions





Forward and backward propagation

$$
\begin{aligned}
Z^{[1]} &= W^{[1]}X + b^{[1]} \\
A^{[1]} &= g^{[1]}(Z^{[1]}) \\
Z^{[2]} &= W^{[2]}A^{[1]} + b^{[2]} \\
A^{[2]} &= g^{[2]}(Z^{[2]}) \\
&\vdots \\
A^{[L]} &= g^{[L]}(Z^{[L]}) = \hat{Y}
\end{aligned}
$$

$$
\begin{aligned}
dZ^{[L]} &= A^{[L]} - Y \\
dW^{[L]} &= \frac{1}{m} dZ^{[L]} A^{[L]T} \\
db^{[L]} &= \frac{1}{m} np.\text{sum}(\text{dZ}^{[L]}, axis = 1, keepdims = True) \\
dZ^{[L-1]} &= dW^{[L]T} dZ^{[L]} g'^{[L]}(Z^{[L-1]}) \\
&\vdots \\
dZ^{[1]} &= dW^{[L]T} dZ^{[2]} g'^{[1]}(Z^{[1]}) \\
dW^{[1]} &= \frac{1}{m} dZ^{[1]} A^{[1]T} \\
db^{[1]} &= \frac{1}{m} np.\text{sum}(\text{dZ}^{[1]}, axis = 1, keepdims = True)
\end{aligned}
$$

Also see hyperparameters n parameters.